

Architecture decision

LiveKit orchestration with existing Dialogflow CX and Amazon Lex skills

Feasible, with conditions. Put the conversation owner in a LiveKit Agents application, retain existing skills behind a common interface, and use A2A where a compatible agent service provides a practical interoperability benefit.

What the design changes

The review asks for one application to understand the caller, select skills, retain context and combine results across platforms. The original audio bridge sent the conversation to CX. In this target, CX becomes a delegated skill backend. This changes application ownership and requires more engineering than adding one external A2A function call. [R0, R1]

Component	Responsibility in the target
LiveKit RTC / SIP	Call ingress, rooms, participants, media transport and delivery.
LiveKit Agents application	Overall conversation, intent queue, authorized skill routing, interruptions and spoken responses.
Common skill interface + adapters	Typed inputs, backend session mapping, task state, deadlines, result validation and continuation.
Existing CX / Lex skills	Skill-specific dialog, slot collection and existing fulfillment. They return control and outcomes to the application.
Application state repository	Permitted context, transcript / summary policy, active workflows and backend identifiers. Durable storage is a separate responsibility.
Human service platform	Queue, desktop, human acceptance and operational handling after handoff. Vendor and method remain open.

What can be reused

Retain working business rules, fulfillment services and bot skills. Reuse the chosen LiveKit transport and supported voice/model plugins. Add the conversation controller, skill contract, adapters and state mapping around those assets. A bot may need a controlled entry route, configured event or fulfillment API exposure before it can serve as a reusable skill. [R7-R9]

The review transcript records that the earlier voice path had not been demonstrated with the required service integrations. This package is a design baseline; it does not establish that the new voice path or a live human transfer has passed an end-to-end test. [R0]

End-to-end call behavior

Read with Views 02, 04 and 05: ingress, conversation, skill execution and return audio.

Step	Flow and owner
1. Admit the call	SIP dispatch places a phone caller in a room, or an authenticated browser joins through WebRTC. The worker attaches to the call and establishes application context.
2. Understand the turn	The baseline pipeline transcribes speech. The application applies identity / access checks, identifies one or more requests and retains any pending intent.
3. Select a skill	The router chooses an approved skill and validates required parameters. The skill interface maps the call / workflow to the correct backend session.
4. Execute	CX receives text through detectIntent; Lex receives text through RecognizeText. An A2A or MCP adapter is used only for the matching service contract.
5. Clarify or wait	If a skill needs input, preserve its continuation. The application asks the question, receives speech or DTMF and resumes that workflow. For ongoing work, monitor status within the call budget.
6. Combine results	Validate each result and retain its source. Process another pending skill when appropriate. Parallel reads are allowed only when independent and session-safe.
7. Speak	The application selects the reply. TTS generates audio, RoomIO publishes it and the LiveKit room delivers it to the caller. CX / Lex audio is not used in this baseline.
8. Continue or close	Handle interruption, recovery or escalation. Persist only permitted context; record the outcome and close sessions / tasks according to policy.

Multi-skill example

The caller asks for claim status and benefit details. CX requests a claim ID; the application collects it and resumes the same CX workflow while retaining the benefit request. After the claim result, the Lex skill returns benefit data. The application presents both sourced results. Any coverage or eligibility determination requires an authoritative rule or service; it must not be inferred by the language model.

Voice ownership

VAD and turn handling are configured in the worker / AgentSession. Barge-in stops eligible playback; business-task cancellation is a separate decision. A realtime speech model is an alternative pipeline, with its own context and recovery behavior. It is not a promise of seamless switching between arbitrary providers. [R2, R3, R13]

Interfaces, state and authentication

Read with View 03. SkillPort is an application contract, independent of the selected backend protocol.

Boundary / API	Transport, configuration and access
LiveKit media and control	WebRTC / DTLS-SRTP media and signalling; SIP / RTP or secured variants at the carrier edge. Configure LiveKit URL, server credentials, room / dispatch policy and SIP trunk where required. Client tokens must be scoped. [R4]
Google Dialogflow CX	Sessions detectIntent via gRPC / TLS or REST / HTTPS. Configure project, location, agent, environment if used, language and session mapping. Use Google application credentials / workload identity and scoped IAM, not a generic CX API key. [R7, R8]
Amazon Lex V2	RecognizeText over HTTPS, signed with AWS SigV4. Configure region, bot ID, alias, locale and session mapping; authorize runtime access through IAM. This API is not gRPC. [R9]
A2A agent service	Discover from an approved card / registry; validate version, binding, skill and auth. Cache cards outside the turn where possible. Use message / task / status / cancellation operations supported by that version. [R10]
MCP tools / direct API	Use the selected MCP transport or typed HTTPS API. Map tool results explicitly; MCP does not by itself provide the same task lifecycle as A2A. [R11]
Voice models / human / telemetry	Selected model APIs need their provider identity. Human desktop / queue integration needs a separate vendor API. Telemetry exports through OTLP to the selected collector and backend. [R2, R6, R12]

Contract and session mapping

Proposed operations: **invoke, resume, status and cancel**. Inputs carry skill/version, typed parameters, caller-authority reference, call/turn/request IDs, deadline and an idempotency key for side effects. Outputs carry **COMPLETED, NEEDS_INPUT, WORKING, FAILED or CANCELLED**, plus validated data, a controlled prompt, continuation and error details. These status names are internal, not A2A wire constants.

Key state by tenant, call, skill and workflow instance. Map that key to the CX or Lex session, or the A2A context/task identifiers. Serialize mutations to one backend session. Define expiry and restart behavior; reject a response belonging to an obsolete turn. Cancelling an RPC does not undo a completed business action.

Keep credentials server-side. Limit data sent to each skill, validate all returned content and allowlist remote endpoints. Authentication of the application to a backend does not replace authorization of the caller.

Feasibility and architectural tradeoffs

The largest uncertainties are skill reuse and live-call behavior, rather than the ability to make an HTTP request.

Dimension	Assessment and condition
Technical feasibility	High at the platform level. The target requires custom routing, state, adapters and failure control; existing cloud integrations still need to be demonstrated together.
Existing skill reuse	Conditional. Confirm each entry route, required slots, completion signal and session behavior. Whole conversational bots may need adaptation or direct fulfillment access.
Latency and call quality	Material risk. Measure routing, backend work and first returned audio under realistic load and network conditions. A fixed 2-3 second deadline is not suitable for every task.
Security and operations	Moderate to high complexity across cloud identities, endpoint trust, sensitive call context, cancellation, provider outages and human routing.
Cost and effort	Greater than the original client-only A2A addition. Cost includes voice transport, models, bot usage, worker capacity, state / telemetry and operations. A credible estimate needs concurrency, call duration and provider choices.
Local feasibility	The worker and test fixtures can run on a developer machine. A real cloud voice demonstration still depends on reachable LiveKit, CX / Lex, configured skills and credentials. Offline fixtures do not prove cloud behavior.

Where A2A adds value

Use A2A when independent agents need a shared discovery and task contract. Native CX / Lex adapters remain valid choices. Exposing existing bots through A2A requires verified native support or a wrapper service with its own deployment and authentication. Do not assume a bot already exposes an Agent Card. [R10]

Latency policy

Measure speech-end to first response audio and task completion separately. Track turn detection, transcription, routing, skill work, synthesis and transport with p50/p95 and end-to-end traces. Streaming may overlap stages. A 3-second added-delay goal for fast lookups is a proposed target, not a result. Long tasks need an agreed wait, supported follow-up or handoff.

Use the simplest compatible adapter for each skill. Keep the application independent of the protocol choice. Adopt A2A when interoperability justifies it.

Operations and review evidence

Read with Views 05 and 06. These are unresolved design inputs and verification needs, not a delivery schedule.

Decision still open	Evidence or owner input needed
Skill inventory and contracts	Identify real CX / Lex skills, owners, supported entry routes, required identity context, outputs and continuation rules. Confirm any native A2A exposure against the deployed service.
Hosting and data location	Select worker placement, LiveKit hosting, service regions and data retention. Confirm cross-cloud network behavior, quotas and active-call capacity.
Model configuration	Select languages, STT / LLM / TTS or realtime model, turn policy and compatible fallback. Measure provider quality and failure behavior on representative calls.
Durable call state	Select state storage, expiry and recovery ownership. A durable summary can preserve business context; it does not automatically reconstruct live audio or move an active call after a worker crash.
Human handoff	Choose the queue and desktop API, supported SIP or room-join method, acceptance signal and recovery path. Context transfer and media transfer must both be verified.

Evidence needed to call the target functional

Demonstrate real voice input, routing, a backend response and audible return. Cover clarification / resume, two skills in one call, simultaneous session isolation, barge-in, required DTMF, service failure and human handoff success / failure. Correlate the calls with traces and measured turn delay. These target scenarios remain unverified.

Failure handling and observability

Bound retries to safe, transient failures. Writes require idempotency and reconciliation of unknown outcomes. Preserve the caller during recovery. A successful cold SIP REFER leaves LiveKit; warm transfer or desktop room join needs an acceptance flow. [R5, R6]

Correlate call, turn, skill and backend task IDs. Record latency, failures, cancellation, usage and transfer outcomes; exclude sensitive content from telemetry. Define application spans and business metrics explicitly, then export through OpenTelemetry. [R12]

Management explanation

“Our LiveKit Agents application owns the conversation and delegates work to existing CX, Lex and other skills. Adapters translate each platform’s format and session behavior. Results return to the application, which asks follow-up questions, combines answers and speaks to the caller. A2A is one integration route. The design is feasible; we still need to demonstrate that our actual skills, voice path and human handoff work together at the required speed.”

Diagram guide and references

Reference date: 12 September 2026. Product capabilities are cited separately from proposed application decisions.

View	Use in the architecture review
01 - Management overview	Explain ownership, reuse, added application responsibilities and the conditional feasibility decision.
02 - Voice protocols	Follow media in and out; show the SIP, RTC and backend API boundaries.
03 - Adapters	Explain the stable contract, provider options and backend translation responsibilities.
04 - Conversation sequence	Walk through clarification, session continuation and two skills in one call.
05 - Exceptions and handoff	Explain barge-in, task failure, late results and human acceptance.
06 - Deployment and telemetry	Show trust boundaries, runtime components, identities and trace collection.

Requirements source

R0. Architecture review transcript, 11 Sep 2026 - supplied as “Pasted markdown.md”. Lines 135-159 establish orchestration ownership and reuse; 183-185 request protocol boundaries and human escalation; 193-207 describe voice verification limits; 211-229 request protocol abstraction and observability.

Official technical references

- R1.** [LiveKit AgentSession](#)
- R2.** [LiveKit model integrations](#)
- R3.** [LiveKit turns and interruptions](#)
- R4.** [LiveKit telephony](#)
- R5.** [LiveKit DTMF and transfers](#)
- R6.** [LiveKit call transfers](#)
- R7.** [Dialogflow CX detectIntent](#)
- R8.** [Dialogflow CX sessions](#)
- R9.** [Amazon Lex V2 RecognizeText](#)
- R10.** [A2A specification](#)
- R11.** [MCP architecture](#)
- R12.** [OpenTelemetry Collector](#)
- R13.** [LiveKit fallback strategies](#)
- R14.** [LiveKit chat context](#)

Icons: official LiveKit symbol, Amazon Lex icon and Google Cloud Agents category icon for CX. See `ICON_SOURCES.md` for provenance. Product marks belong to their owners.